

Q1. Fraud Transaction Detection (Linear Search)

```
#include <iostream>
using namespace std;

int main()
{
    int N;

    cout << "Enter number of transactions: ";
    cin >> N;

    int transactions[N];

    cout << "Enter transaction IDs: ";
    for(int i = 0; i < N; i++)
    {
        cin >> transactions[i];
    }

    int suspiciousID;

    cout << "Enter suspicious transaction ID: ";
    cin >> suspiciousID;

    bool found = false;

    for(int i = 0; i < N; i++)
    {
        if(transactions[i] == suspiciousID)
        {
```

```
found = true;
```

```
break;
```

```
}
```

```
}
```

```
if(found)
```

```
cout << "\nFraud Transaction Found";
```

```
else
```

```
cout << "\nTransaction Not Found";
```

```
return 0;
```

```
}
```

Q2. Server Log Search (Binary Search)

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int N;
```

```
cout << "Enter number of log files: ";
```

```
cin >> N;
```

```
int logs[N];
```

```
cout << "Enter sorted log files: ";
```

```
for(int i = 0; i < N; i++)
```

```
{  
cin >> logs[i];  
}
```

```
int target;
```

```
cout << "Enter target log AB: ";  
cin >> target;
```

```
int low = 0;  
int high = N - 1;  
bool found = false;
```

```
while(low <= high)  
{  
int mid = (low + high) / 2;
```

```
if(logs[mid] == target)  
{  
found = true;  
break;  
}
```

```
else if(logs[mid] < target)  
{  
low = mid + 1;  
}
```

```
else  
{  
high = mid - 1;  
}  
}
```

```
if(found)
cout << "\nLog Found";
else
cout << "\nLog Not Found";

return 0;
}
```

Q3. Sort Employee Salaries (Bubble Sort)

```
#include <iostream>
using namespace std;

int main()
{
int N;

cout << "Enter number of employees: ";
cin >> N;

int salary[N];

cout << "Enter salaries: ";
for(int i = 0; i < N; i++)
{
cin >> salary[i];
}

for(int i = 0; i < N - 1; i++)
```

```

{
for(int j = 0; j < N - i - 1; j++)
{
if(salary[j] > salary[j + 1])
{
int temp = salary[j];
salary[j] = salary[j + 1];
salary[j + 1] = temp;
}
}
}

```

```

cout << "\nSorted Salaries: ";

```

```

for(int i = 0; i < N; i++)
{
cout << salary[i] << " ";
}

```

```

return 0;
}

```

Q4. Sort Website Response Time (Insertion Sort)

```

#include <iostream>
using namespace std;

```

```

int main()
{

```

```
int N;
```

```
cout << "Enter number of response times: ";
```

```
cin >> N;
```

```
int responseTime[N];
```

```
cout << "Enter response times: ";
```

```
for(int i = 0; i < N; i++)  
{
```

```
    cin >> responseTime[i];  
}
```

```
for(int i = 1; i < N; i++)  
{
```

```
    int key = responseTime[i];  
    int j = i - 1;
```

```
    while(j >= 0 && responseTime[j] > key)  
    {  
        responseTime[j + 1] = responseTime[j];  
        j--;  
    }
```

```
    responseTime[j + 1] = key;  
}
```

```
cout << "\nSorted Response Times: ";
```

```
for(int i = 0; i < N; i++)  
{
```

```
cout << responseTime[i] << " ";  
}
```

```
return 0;  
}  
...
```

```
```cpp
```

Q5. Prioritize Bug Reports (Selection Sort)

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int N;
```

```
cout << "Enter number of bug priorities: ";
```

```
cin >> N;
```

```
int priority[N];
```

```
cout << "Enter priority scores: ";
```

```
for(int i = 0; i < N; i++)
```

```
{
```

```
cin >> priority[i];
```

```
}
```

```
for(int i = 0; i < N - 1; i++)
```

```
{
```

```
int minIndex = i;
```

```

for(int j = i + 1; j < N; j++)
{
 if(priority[j] < priority[minIndex])
 {
 minIndex = j;
 }
}

```

```

int temp = priority[i];
priority[i] = priority[minIndex];
priority[minIndex] = temp;
}

```

```

cout << "\nSorted Priority Scores: ";

```

```

for(int i = 0; i < N; i++)
{
 cout << priority[i] << " ";
}

```

```

return 0;
}

```